

The class GridCollection

Manual (draft)

Alex Pfaff
nonintersective@gmail.com

December 9, 2025

Contents

1	Initialization and Central Properties	3
1.1	Constructor	3
1.2	Attributes	5
1.3	Checking methods	5
2	Activation → “Active Grids”	6
2.1	Vertical Series	6
2.2	Horizontal Series	7
2.3	THIS Series	7
2.4	Multi-Class Series	9
2.5	Puzzle Series	10
3	The Garbage Collection	13
4	Create train/test Datasets	14

Welcome to Sudoku and Congratulations on your acquisition of `GridCollection`! – it reveals your good taste in sophisticated cool stuff, highlights your fearlessness in the face of structural insanity and combinatorial explosions, and thus betrays your non-standard curiosity towards the unknown (depths of Sudoku geometry); well done!

`GridCollection` is a class that manages collections of Sudoku grids, it allows you to “activate” various categories of sets of Sudoku grids (e.g. permutation series), encode or manipulate them in various ways, and create (train/test) datasets for various tasks, notably, to train ML models / neural networks (even though the original purpose of this endeavour is more academic in nature: Sudoku as a formal language – the grammar of Sudoku grids). I will occasionally comment on some such practical practical aspects. One noteworthy feature is the option to manufacture your own (customized) garbage collection; e.g. for classification tasks, you can determine certain properties of the data that will receive the label `False` – awesome, right?

This manual illustrates the most important features / uses of the `GridCollection` class; there is also some extensive documentation in the code (docstring, use the Python built-in function `help()` to access the documentation). Feel free to drop me a line (feedback, questions, bug reports, suggestions, comments on Sudoku geometry ...).

Best wishes, Alex

PS: this manual is “work in progress” to use an academic phrase; certain relevant aspects have not been covered yet. Be patient, or – as already stated above – feel free drop me a line.

1 Initialization and Central Properties

1.1 Constructor

The class `Grid` has a method `permuteGrids(toCollection = True)` – that generates all 1679616 grid permutations from the current grid ($\rightarrow$ the vertical series), and, by default, returns a `GridCollection` object. Likewise, the method `get_minTripletCollection(to_gc=True)` returns a `GridCollection` object.

The class `GridCollection` itself has a number of methods for initialization:

(1) Static constructor method (generation from scratch; recommended):

```
>>> gc = GridCollection.from_scratch()
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296

>>> print(gc)
GridCollection[
    internal shape: (1679616, 81)
    active series: not_activated
        size: 0
    labeled series: 0
    labels: False
        size: 0
    garbage: False
    total size: 0
        guest_grids                : 0
        false_fromCurrent_seq      : 0
        false_fromCurrent_switch  : 0
        false_cardinality          : 0
        false_off_by_X             : 0
        false_arbitrary            : 0
    one_hot encoding: False
    internal abc collection: False
]
```

This method generates a (valid) Sudoku grid g at random, and then internally calls `g.permuteGrids()`. Thus **the default grid collection is a collection of grid permutations** (= vertical permutation series). The method allows the option to initialize a specific grid via the setting `GridCollection.from_scratch(grd = myGrid)` where `myGrid` is a valid Sudoku grid (as array or sequence type: tuple, list, string).

(2) Proper constructor method:

```
>>> gc = GridCollection(gridArray)
>>> # with gridArray.shape == (n, 81) or (n, 9, 9); n = number of grids
>>> # with gridArray.dtype == int or 'U1' (str)
```

This method presupposes that the user has already a suitable collection of (valid!) Sudoku grids (as `np.ndarray`) of an appropriate shape and dtype. Moreover, the user bears responsibility for the validity of the grids contained!

(3) Static constructor method (from sql database):

```
>>> gc = GridCollection.from_sql(db_name, how_many = None)
```

This method presupposes a suitable/valid grid collection (see (2)) stored in an sql database by an appropriate method (see Sect. XXX). `db_name` specifies the name of the db/file, and `how_many` specifies how many grids are to be loaded; plausibly, the number specified must not be greater than the size of the database. If set to `None` (default), the entire database is loaded into the `GridCollection` object.

(4) Diverse Static constructor method (experimental stage!):

```
>>> gc = GridCollection.from_scratch_div(how_many = 500000,
                                         series = 10,
                                         seed = 42)
```

```
Generating guest grids, series 1:
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296
Dialecting grid collection: 100%| 1679616/1679616
Current size: 50000
Generating guest grids, series 2:
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296
Dialecting grid collection: 100%| 1679616/1679616
Current size: 100000
```

.

```
Generating guest grids, series 9:
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296
Dialecting grid collection: 100%| 1679616/1679616
Current size: 450000
Generating guest grids, series 10:
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296
Dialecting grid collection: 100%| 1679616/1679616
Current size: 500000
Checking for duplicates: 100%| 500000/500000
Generating missing grids: 100%| 3359232/3359232
Adding missing grids: 100%| 3699/3699
```

```

>>> print(gc)
GridCollection[
    internal shape: (500000, 81)
    active series: not_activated
    size: 0

    . . . .
]

```

1.2 Attributes

As illustrated above, the current status of the collection object can be displayed via print statement `print(gc)`, alternatively, via the method `gc.getInfo()`.

```

>>> gc.BASE_NUMBER
3
>>> # 3 is the basenumber for classical Sudoku grids;
>>> # presupposed for most functionalities
>>> gc.DIMENSION # = gc.BASE_NUMBER^2
9
>>> gc.SIZE_grid # = gc.DIMENSION^2
81

>>> gc.SIZE_collection
>>> gc.SIZE_activeSeries
>>> gc.SIZE_garbageCollection
>>> gc.SIZE_labels

>>> gc.gc.activationStatus

>>> gc.activeSeries

>>>

....

```

1.3 Checking methods

2 Activation → “Active Grids”

The internal collection (from which the object was initialized) is inert, as it were; it needs to be *activated*.¹ Based on the `activeSeries`, various manipulations can be conducted and datasets (including garbage collections) can be built. Therefore, activation is the crucial next step after initialization; the class `GridCollection` provides a range of different activation methods to be illustrated below (notice, that in some cases, the garbage collection must be initialized before activation!).

2.1 Vertical Series

```
>>> gc.activate_verticalSeries()

>>> print(gc)
GridCollection[
    internal shape: (1679616, 81)
    active series: vertical
    size: 1679616
    . . . . .
]
```

If the object is initialized via `gc = GridCollection.from_scratch()` (or e.g. `gc = g.permuteGrids()`), the internal collection is a vertical series, and will be activated directly (see fn. 1).

If the object is initialized differently and the internal collection is not a collection of (1679616) grid permutations, an exception is raised suggesting two possible fixes:

```
>>> # produces collection of triplet grids, not row/col permutations!
>>> gc = g.get_minTripletCollection(to_gc=True)
>>> print(gc)
GridCollection[
    internal shape: (373248, 81)
    ....

>>> gc.activate_verticalSeries(explicit_validation=True) # default setting
...
ValueError: Collection does not resemble a vertical series.
Use '.activate_thisSeries()' instead
-- or specify 'new_series_fromIndex' parameter.
```

¹The original purpose of the class `GridCollection` was to manage large collections of grid permutations. For this reason, vertical and horizontal permutation series still play a major role in various activation and generation procedures / methods (this will be externalized and more delegated in later versions).

If the user wants to activate the internal collection nonetheless (i.e. regardless of provenance / contents), there is a designated method doing just that; see Sect. XXX

If the user insists on activating a vertical series, however, the method provides a parameter $0 \leq \text{new_series_fromIndex} < \text{gc.SIZE_collection}$, that takes the specified grid from the internal collection, and generates a vertical series from scratch $\rightarrow$ from which, in turn, the active series initialized:

```
>>> gc.activate_verticalSeries(new_series_fromIndex=42)
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296

>>> print(gc)
GridCollection[
    internal shape: (373248, 81)
    active series: vertical_new
    size: 1679616
    . . . .
]
```

2.2 Horizontal Series

Differently from the vertical permutation series, which is the (exhaustive) collection of grid permutations obtained from row/column permutations, the horizontal permutation series is obtained from label permutations; this is achieved by exhaustive label recoding. Any classical 9×9 Grid can be recoded in $9! = 362,880$ ways, and all those grids translate to the same abc-grid, i.e. they have the same (deep) distribution. The activation method takes an index argument (default: $\text{idx} = 0$), with $\text{idx} \leq 0 < \text{gc.SIZE_collection}$, which in turn, picks the corresponding grid from the internal collection that serves as the basis for the horizontal permutations:

```
>>> gc.activate_horizontalSeries(idx = 666) # grid 666 -> basis

>>> print(gc)
GridCollection[
    internal shape: (1679616, 81)
    active series: horizontal
    size: 362880
    . . .
]
```

2.3 THIS Series

If the internal collection does not constitute a vertical series, but the user wants to activate it, THIS is the method of your choice:

```
>>> gc.activate_thisSeries(name = 'THIS')
```

```
>>> print(gc)
GridCollection[
    internal shape: (1679616, 81)
    active series: THIS
    size: 1679616
    . . . .
]
```

‘THIS’ is the default label, but obviously, it can be customized (`name = 'my_series13'`); (`name = 'my_lucky_grids'`), ideally in a way that actually says something about the provenance/properties of the collection. A suitable use for the scenario mentioned in Sect. 2.1 is illustrated below:²

```
>>> gc = g.get_minTripletCollection(to_gc=True)
>>> gc.activate_thisSeries(name='3-triplets')

>>> print(gc)
GridCollection[
    internal shape: (373248, 81)
    active series: 3-triplets
    size: 373248
    . . . .
]
```

Likewise, when using the diverse constructor, the following labeling may be useful:

```
>>> gc = GridCollection.from_scratch_div()
. . . .
>>> gc.activate_thisSeries(name='diverse') # or name = 'random' etc.

print(gc)
GridCollection[
    internal shape: (500000, 81)
    active series: diverse
    size: 500000
    . . . .
]
```

NB: the class `GridCollection` also has a method `activate_randomSeries()` that, in essence, does the same thing. But it should be considered deprecated and will be removed in a future version.

²For more information on *triplets*, see “Grid_class_manual”, Sect. 3.2.

2.4 Multi-Class Series

For multiclass classification (and other creative) tasks, this method is your friend. A few words are in order, however, about some noteworthy peculiarities. Firstly, classes are constituted by horizontal series (based on a random selection of grids from the internal collection), meaning that each class comprises exactly 362,880 members. In the future it will be possible base classes on other features and (geometric) properties, but notice that any class(ification) of grids that can reasonably justified will be a finite set and the procedure will generate an exhaustive dataset. This is the motivation, secondly, for the inclusion of a garbage class = everything that is not a member of our actual classes. The method has the recommended default setting (`make_garbage=True`), which therefore and thirdly presupposes that the garbage collection has been initialized to an appropriate size (min: 362,880; the garbage collection will be discussed in detail in Sect. CCC):

```
>>> # if garbage size < 362880, an exception will be raised
>>> gc.makeFalseGrids_cardinality(how_many=362880)

>>> gc.activate_multiClassSeries(classes=5)

>>> print(gc)
GridCollection[
    internal shape: (500000, 81)
    active series: multiclass
    size: 2177280
    labeled series: 6 classes
    labels: True
        size: 2177280
    garbage: True
        total size: 362880
        guest_grids           : 0
        false_fromCurrent_seq : 0
        false_fromCurrent_switch : 0
        false_cardinality     : 362880
        false_off_by_X        : 0
        false_arbitrary       : 0
    one_hot encoding: False
    internal abc collection: False
]
```

The string representation finally reveals the fourth peculiarity: even though 5 classes are specified in the activation method (`classes=5`), we find `labeled series: 6 classes`. This discrepancy stems from the inclusion of the garbage class; therefore (size) $2177280 = 6 \text{ (classes)} \times 362880$. The garbage class always receives the label 0, whereas the actual classes receive the labels 1–5 (or whatever number specified) in an arbitrary fashion (it's hard to see what meaningful labels should be assigned

otherwise).

2.5 Puzzle Series

The puzzle series constitutes a dataset suitable to train a (DL) model to solve Sudoku grids (and other creative tasks). More specifically, it creates a dataset on the basis of the currently active series. Here is the method signature:

```
activate_puzzleSeries(k: int,  
                      extra_high_vals: bool = True,  
                      include_parentSet: bool = True,  
                      run: int = 3  
                      ) -> None
```

The only mandatory parameter is k , which specifies the (maximum) number of blanks; put differently, $81 - k$ = number of given digits, indicating the degree of difficulty of the grids to be solved. Let's elaborate on that point: since the required minimum number of given digits is 17, it is not recommended to set $k > 64$. Moreover, regardless of what machines and models can actually do, one does not play $\rightarrow$ solve a Sudoku grid by filling in all the blanks in one go, where would be the thrill in that? No, we proceed cautiously and cunningly one cell at a time. For this reason, k is not a rigid parameter, but an upper bound; the underlying algorithm generates an equal number of grids with $n = 1$ blank, $n = 2$ blanks ... $n = k - 1$ blanks, $n = k$ blanks, respectively. For a given n and a given grid currently being iterated over, the positions for the n blanks are randomly selected.

In order to see what a data point (blanked grid, label) looks like, let's first do some actual activation:

```
>>> gc.activate_puzzleSeries(k=55)  
Generating grids with 1-55 blanks: 100%||  
Generating grids with 35-55 blanks: 100%||  
  
>>> print(gc)  
GridCollection[  
    internal shape: (373248, 81)  
    active series: k_55_max_puzzle_from_3-triples  
        size: 2612736  
    labeled series: 55 k_blanks  
    labels: True  
        size: 2612736  
  
    . . .  
]
```

```

>>> # active series is now a collection of blanked grids
>>> # with n = 1-k blanks
>>> # blanks are represented by zeros
>>> gc.activeSeries[654321].reshape(9,9)

array([[0, 2, 3, 0, 5, 6, 0, 0, 9],
       [0, 6, 4, 8, 9, 7, 0, 3, 1],
       [7, 0, 0, 2, 3, 1, 5, 6, 4],
       [3, 1, 2, 0, 0, 4, 9, 7, 0],
       [6, 0, 5, 7, 8, 9, 0, 0, 2],
       [8, 9, 0, 3, 1, 2, 0, 4, 0],
       [2, 3, 1, 0, 0, 0, 8, 9, 7],
       [4, 5, 6, 9, 7, 8, 0, 2, 0],
       [9, 7, 8, 1, 0, 3, 0, 5, 6]], dtype=uint8)

>>> # the labels are complete grids with no blanks!
>>> # the following is the label for the blanked grid above
>>> gc.labels[654321].reshape(9,9)

array([[1, 2, 3, 4, 5, 6, 7, 8, 9],
       [5, 6, 4, 8, 9, 7, 2, 3, 1],
       [7, 8, 9, 2, 3, 1, 5, 6, 4],
       [3, 1, 2, 5, 6, 4, 9, 7, 8],
       [6, 4, 5, 7, 8, 9, 3, 1, 2],
       [8, 9, 7, 3, 1, 2, 6, 4, 5],
       [2, 3, 1, 6, 4, 5, 8, 9, 7],
       [4, 5, 6, 9, 7, 8, 1, 2, 3],
       [9, 7, 8, 1, 2, 3, 4, 5, 6]], dtype=uint8)

```

As shown, a label is not simply a list (vector) of the missing values for the blanks, but a complete grid. The problem with the *label = list-of-missing-values strategy* is that it entails a specified output size ($= \text{len}(\text{label})$). This, in turn, is at odds with the variable parameter $n = 1, 2 \dots k$: we would need k different datasets of different sizes and train k different models, one for each value n . So in spite of a slight overhead, the alternative of using a complete grid (the "solution" to a blanked grid) as a label is a pretty nifty and elegant way to avoid a logistic and computational nightmare :)

Back to the parameters: `extra_high_vals` is by default set to `True`; in this case an additional set of blanked grids is generated for $n = \lfloor \frac{2k}{3} \rfloor \dots k-1, k$. Intuitively, grids with a larger number of blanks tend to be more "difficult/complex" than grids with a smaller number of blanks, at least combinatorially: for $n = 1$, there are 81 possible placements for the one blank, but with increasing n , the number of possible (combinations of) placements grows super-exponentially: $\binom{81}{n}$. In other words, for larger n , there will be less and less ^{n} actual occurrences of possible placements of blanks in the dataset. This parameter is a modest attempt to slightly ameliorate this

combinatorial inequality.

The parameter `include_parentSet`: if set to `True` (default), the entire (original) active series is added to the puzzle series, with 0 blanks; in these cases, the “blanked” grids are identical to their respective labels.

The parameter `run` specifies the number of iterations over the currently active series; the size of the puzzle series is thus

```
run × gc.SIZE_activeSeries
    × 2 for extra_high_vals
    + gc.SIZE_activeSeries, if include_parentSet=True
```

The following simplified code snippet illustrates what happens during a run:

```
>>> for _ in range(run):
    for idx in range(gc.SIZE_activeSeries):
        n = idx % k + 1 # -> values from 1 to k
        grid = gc.activeSeries[idx]
        labels.add(grid)
        # modify the grid:
        # select n positions in grid at random
        # replace the values in those positions with 0
        # <--> 'blank' them
        ....
        puzzles.add(grid)
    ...
```

Every grid from the original active series occurs *run* times as a blanked grid in the puzzle series. It is conceivable that the same grid occurs *run* times with the same number of *n* blanks (unlikely in the same positions!), but more likely, with different values *n*. But even if $run \geq k$, it is very extremely unlikely that the same grid occurs with all values $n \in [k]$, and it's virtually impossible that the puzzle series contains a complete, consecutive solution path for even a single grid (from *k* to 0 blanks).

Obviously, increasing *run* leads to more diversity in the dataset (possible combinations of blank positions in the grid), but even more obviously, it leads to a larger dataset. This has various implications for the runtime on a household computer. BUT consider this: this method allows you to **create an insanely large high-quality labeled dataset**. Obviously the kind and size of the current active series do play a role, but the range of combinatorial possibilities almost guarantees that the dataset won't get repetitive no matter how much you increase *run*.

Remember: Only one single grid gives rise to $\sum_{n=1}^k \binom{81}{n}$ blanked grids!

PS: it's not a violation of any rules if you want to create a puzzle series only to solve them yourselves – good luck and enjoy ;)

3 The Garbage Collection

4 Create train/test Datasets